ATTS: Agentic Test-Time Scaling via Adaptive Exploration

Peijia Qin
University of California, San Diego
pqin@ucsd.edu

Abstract

Test-time scaling has become a key mechanism for improving large language model reasoning, but existing approaches often rely either on rigid heuristic allocation rules or on explicit fine-grained search over intermediate reasoning steps. We introduce ATTS, a delegated sequential test-time scaling framework in which a lightweight orchestrator adaptively controls a pool of candidate solutions through a single action: `explore`, which launches a fresh, independent solver. The orchestrator decides when to stop exploring and directly synthesizes the final answer from the accumulated evidence. By restricting control to this minimal interface, ATTS shifts compute allocation toward semantic evidence management over candidate hypotheses. We evaluate ATTS on six benchmarks spanning mathematical reasoning, scientific question answering, code generation, and multimodal reasoning. ATTS outperforms or matches all baselines across the four main benchmarks while remaining cost-competitive: 82.83% on GPQA-Diamond, 82.29% on LiveCodeBench, 56.00% on HLE-Verified, and 23.20% on BabyVision. An ablation replacing the orchestrator's direct synthesis with a separate integrator call degrades accuracy on three of four benchmarks, indicating that stateful evidence accumulation during exploration is critical to the method's effectiveness.

1 Introduction

The allocation of inference-time compute has emerged as a central frontier for improving the reasoning capabilities of large language models. Recent work on test-time compute is commonly organized around sequential scaling, parallel scaling, and hybrid search. The compute-allocation perspective of Snell et al. [24] makes clear that no single strategy is uniformly optimal across prompts of varying difficulty. At the same time, simply allocating more compute is not enough: naive scaling can saturate, and in sampling-based settings can even degrade through poor coverage and reward hacking [6]. The central question is therefore not whether to scale test-time compute, but how to allocate it adaptively and efficiently.

Existing approaches often answer this question with either heuristics or explicit micro-level search. Predictive routing methods such as DiffAdapt use lightweight difficulty estimates to choose an inference regime before generation begins. However, these decisions remain open-loop and cannot react to evidence that appears during reasoning [12]. At the other extreme, Tree of Thoughts and process-reward frameworks such as Math-Shepherd perform closed-loop adaptation through explicit tree expansion, pruning, and step-level verification [33, 29]. Recent agentic test-time scaling methods add another variant of this pattern, but many still rely on statistical control rules such as vote entropy, confidence-triggered halting, or parallel mixtures of tool-using agents [9, 5, 10]. An underexplored alternative is a lightweight orchestrator that allocates compute over a semantic pool of candidate hypotheses without explicit search trees or handcrafted stopping thresholds.

Preprint. Under review.

We introduce ATTS, a delegated sequential test-time scaling framework that separates *control* from *solution generation*. A lightweight orchestrator manages a growing pool of candidate solutions through a single tool, `explore`, which dispatches a fresh, independent solver on the original problem. When the orchestrator judges the pool to be sufficiently informative, it stops exploring and synthesizes the final answer directly from the accumulated evidence. Echoing Sutton's "Bitter Lesson" [25], our claim is not that richer control languages are necessary, but that a minimal action space can be functionally preferable when compute allocation should depend on semantic evidence rather than brittle heuristics.

We evaluate ATTS across six benchmarks spanning mathematical reasoning, scientific question answering, code generation, and multimodal reasoning: HLE-Verified [34], LiveCodeBench [8], BabyVision [4], AIME 2025 and AIME 2026 [16, 17], and GPQA-Diamond [20]. Because the orchestrator is benchmark-agnostic, the same policy transfers across text-only and image-conditioned tasks.

2 Related Work

Recent work on test-time compute for large language models is commonly discussed under three broad families: sequential scaling, parallel scaling, and hybrid search. Following the compute-allocation perspective of Snell et al. [24], we use this taxonomy only to position ATTS relative to existing inference-time strategies.

Sequential Scaling. Sequential scaling increases reasoning depth by allocating additional computation along a single evolving trajectory. The defining property is causal dependence across steps: later computation conditions on earlier intermediate outputs, so the extra budget is spent on iterative refinement, longer deliberation, or explicit backtracking rather than on independent samples. Representative examples include self-correction and revision methods such as Self-Refine [15], as well as long-thinking or budget-controlled decoding methods such as s1 with Budget Forcing [18]. Recent benchmark work also sharpens the empirical picture of this family: CorrectBench shows that self-correction can help on hard reasoning tasks, but that sophisticated correction pipelines often deliver limited gains relative to their added cost, and that simple CoT baselines remain surprisingly competitive [26]. In this family, test-time scaling is achieved by making one reasoning path deeper, more reflective, or more persistent before committing to a final answer.

Parallel Scaling. Parallel scaling increases reasoning width by drawing multiple independent candidates and then aggregating or reranking them. The classical instance is self-consistency, which samples diverse reasoning paths and marginalizes over their final answers [31]. More recent work pairs this width expansion with learned reward models or verifier models that score full responses, enabling reward-model-based selection rather than simple majority voting. This line includes outcome-reward reranking in the compute-scaling analysis of Snell et al. [24] and modern open reward models designed explicitly for majority voting inference, such as Skywork-Reward-V2 [11]. Recent theoretical work further shows that naive majority voting scaling is not uniformly monotonic: its effectiveness depends on policy coverage over good responses, and overly aggressive scaling can degrade performance through reward hacking [6]. In a complementary line of work, Chen et al. [3] argue that this issue is not only an inference-time phenomenon but also a training-time one, showing that standard cross-entropy fine-tuning can become misaligned with pass@N scaling by making models overconfident. In multimodal settings, the same pattern appears in VisualPRM, which scores candidate multimodal reasoning traces for reranking [30]. The common structure is that candidates are generated independently and the additional compute is spent on breadth plus post hoc selection.

Hybrid Search. Hybrid methods combine depth and width by repeatedly expanding multiple partial trajectories and pruning them with search or reward signals. Tree of Thoughts is an early and influential example: it generalizes chain-of-thought into a tree over intermediate "thoughts," allowing lookahead, self-evaluation, and backtracking [33]. Subsequent work strengthens this paradigm with more explicit search and value estimation, for example by integrating Monte Carlo Tree Search with iterative preference learning [32] or by introducing step-level process reward models such as Math-Shepherd to provide dense supervision over intermediate reasoning steps [29]. More recent hybrid systems such as Recursive Self-Aggregation explicitly combine population-level candidate

aggregation with iterative refinement, showing that depth and width can be interleaved rather than treated as disjoint scaling axes [28]. These methods occupy the middle ground between pure sequential refinement and pure parallel sampling. We position ATTS against this landscape as a delegated sequential orchestrator rather than an explicit tree-search method.

Agentic Test-Time Scaling. Recent work has begun to study test-time scaling explicitly for agents. CATTS allocates compute using vote-derived uncertainty statistics such as entropy and top-1/top-2 margin [9]; TUMIX scales through parallel heterogeneous agents with confidence-based early stopping [5]; and General AgentBench provides a broader empirical study of sequential and parallel scaling for open-ended LLM agents [10]. These works establish that agent settings introduce new scaling bottlenecks, but they predominantly rely on statistical routing rules or parallel mixtures. In contrast to these designs, ATTS focuses on reasoning benchmarks where the central object of control is a semantic pool of candidate hypotheses, and where adaptive compute allocation is expressed through a single action—`explore`—rather than explicit search trees or handcrafted uncertainty thresholds.

3 Methodology

3.1 Problem Formulation

Given an input problem x, the goal is to produce a final answer y under a bounded test-time budget. ATTS maintains a state $S_t = (x, \mathcal{C}_t, b_t)$, where $\mathcal{C}_t = \{c_1, \ldots, c_{n_t}\}$ is the current candidate pool and b_t is the remaining exploration budget. Each candidate $c_i = (a_i, r_i, p_i, s_i)$ contains an answer, free-form reasoning, an approach summary, and an optional self-reported confidence score.

3.2 The ATTS Loop

The method proceeds as follows. A lightweight orchestrator repeatedly decides whether to call `explore` or to stop and synthesize. It never solves the problem itself.

Explore. Each `explore` call dispatches a fresh solver on the original problem x. The solver works from scratch—it does not see or continue previous candidates—and returns a structured candidate that is appended to $\mathcal{C}_t$. This fresh-start design encourages diversity: different strategies, error modes, and confidence profiles emerge independently across candidates.

Stopping. The orchestrator can stop at any point before the exploration budget T is exhausted. The stopping decision is made by the orchestrator model itself based on its assessment of the candidate pool; no external stopping criterion or confidence threshold is imposed. The prompt encodes principles for this judgment: genuine convergence means independent solvers arriving at the same answer through different methods, repeated failures indicate the problem exceeds solver capability, and each additional explore call has a cost that must be justified by expected information gain. If the budget is exhausted without an explicit stop, the method falls back to the highest-confidence candidate in the pool.

Synthesis. When the orchestrator stops, it synthesizes the final answer directly from $\mathcal{C}_t$. Because it has observed the full multi-turn history—each candidate's arrival, its reasoning, and its own deliberation about whether to continue—it has richer context than a separate aggregator that only sees the final pool. The orchestrator compares candidates, reconciles partial evidence, and produces a final answer that reflects the aggregate evidence.

The control logic is benchmark-agnostic: only the explorer prompt and answer schema change across tasks. Appendix A gives the full implementation details.

3.3 Prompt Design: Principles Over Instructions

The orchestrator prompt uses a principles-based design rather than prescriptive instructions. Instead of specifying when to explore or stop (e.g. "explore at least twice before stopping"), the prompt states truths about the system (e.g. "a single candidate does not constitute sufficient evidence", "an answer without evidence from candidates is baseless") and provides worked examples showing how these

principles apply in concrete scenarios. This design arose from empirical observation: prescriptive rules led to brittle behavior where the orchestrator would violate instructions in edge cases (such as attempting to solve problems itself when all solvers timed out), while principles-based prompts allowed the model to derive appropriate behavior from its understanding of its role. The examples serve as demonstrations of principle application rather than exhaustive case coverage.

3.4 Rational Metareasoning Perspective

The ATTS orchestrator can be understood through the lens of *rational metareasoning* [21]: rather than solving the problem directly, it allocates computational resources (explore calls) to maximize expected answer quality under a budget constraint. Each explore result updates the orchestrator's *belief state*—its accumulated evidence about which answer is correct—and the next action is chosen based on the current belief. This sequential, memory-dependent structure distinguishes ATTS from memoryless strategies such as Best-of-N, where all candidates are generated independently and no intermediate evidence informs subsequent sampling.

From this perspective, the orchestrator performs *budget-constrained optimization in belief space*. Each explore functions as an information gradient step that reduces uncertainty about the correct answer, and the choice of action configuration (which model, at what cost) determines the step's magnitude and direction. The stopping criterion—whether additional exploration has positive marginal value—is the natural analogue of a convergence check. This framing yields a concrete design principle: the value of the ATTS orchestrator is proportional to the richness of its action space. When the only action is "explore once more with the same model," the orchestrator can only decide *when* to stop. With heterogeneous models, it also decides *what* to compute. As the action space expands further (e.g. solver effort level, time budget, decomposition strategy), the orchestrator gains additional degrees of freedom, enabling finer-grained adaptation to problem difficulty.

3.5 Multi-Model Extension

We extend ATTS to multi-model deployment by adding model selection to the orchestrator's decision space. The orchestrator (Claude Sonnet 4.6) dispatches explores to a pool of three models—Haiku (cheapest), Sonnet (mid-range), and Opus (strongest)—each with a per-question budget limit. Model profiles inform dispatch decisions: the orchestrator starts with cheaper models, escalates on disagreement or repeated failure, and treats cross-model agreement as stronger evidence than same-model convergence. This enables cost-aware exploration strategies where easy problems can be solved entirely with cheap models while hard problems automatically escalate to more capable solvers. An *exploration effort* parameter controls stopping aggressiveness: Low effort permits early stopping when any two candidates converge, Medium effort requires at least three explores across at least two different models, and High effort requires at least five explores using all three models.

4 Experiments

We evaluate on six benchmarks: HLE-Verified [34], LiveCodeBench [8], BabyVision [4], AIME 2025 and AIME 2026 [16, 17], and GPQA-Diamond [20]. The main text presents results on HLE-Verified, LiveCodeBench, BabyVision, and GPQA-Diamond, which collectively span mathematical reasoning, scientific question answering, code generation, and multimodal reasoning. AIME 2025 and AIME 2026 are deferred to Appendix B because the backbone model already achieves near-ceiling accuracy on these 30-problem benchmarks, limiting the statistical resolution for method comparison. Appendix D collects the full official task definitions, grading rules, source repositories, and leaderboard alignment notes for all benchmarks.

4.1 Experimental Setup

Across all experiments, the underlying backbone model is Claude Sonnet 4.6 (`claude-sonnet-4-6`), used for the orchestrator, explorer, and all baselines unless otherwise noted. Claude Haiku, Sonnet, and Opus are three capability tiers in the Claude model family, ordered from smallest/cheapest to largest/most capable. All generation uses the model's default sampling parameters (no explicit temperature or top-p override) with extended thinking enabled at a budget of 32,000 tokens. Extended thinking allows the model to allocate a portion of its output budget to internal chain-of-thought

reasoning before producing the final structured response; all baselines use the same setting for fair comparison. The exploration budget is $T{=}8$ `explore` calls per question for single-model runs and $T{=}16$ for multi-model runs (per-model limits: Haiku 8, Sonnet 8, Opus 4).

We compare against six baselines, all sharing the same backbone model and answer interface so that only the inference-time strategy differs (full details in Appendix A): Pass@1 (single rollout), Majority Voting ($N{=}8$ independent candidates, most-frequent answer; reported only on benchmarks where direct answer aggregation is well-defined), Self-Refine [15] (iterative feedback-revision on a single draft), Budget Forcing [18] (N sequential rounds with forced continuation via "Wait"), reward-model reranking using Skywork-Reward-V2 [11, 23] for text benchmarks or VisualPRM [30, 19] for multimodal benchmarks, and LLM Selection ($N{=}8$), which feeds all pre-cached candidates to the backbone model in a single call and asks it to analyze, compare, and synthesize a final answer. LLM Selection isolates the contribution of LLM-based aggregation from adaptive sequential exploration.

For ATTS, the controller issues up to the exploration budget of `explore` calls before stopping and synthesizing the final answer directly, as described in Section 3. The comparison across all rows isolates differences in test-time inference strategy rather than differences in training data, parameter count, or post-training recipe. In all tables, **bold** marks the best accuracy per group (Zero-Shot Models and Test-Time Scaling Methods separately) and the lowest cost per question among test-time scaling methods.

4.2 Main Results

Table 1 presents the main results across all four benchmarks. For each benchmark, the upper group reproduces published zero-shot references for context, and the lower group reports test-time scaling methods evaluated under our unified setup. Zero-shot references are cited from the respective benchmark papers and official leaderboards; they use different model configurations and are therefore not directly comparable to our test-time scaling rows. HLE-Verified zero-shot rows report Revised-subset Acc [34] while our rows evaluate a 100-sample Gold subset. LiveCodeBench references follow the public generation leaderboard [8]. BabyVision references are from Chen et al. [4]. GPQA-Diamond references are from Rein et al. [20] and official records. Full benchmark details including dataset releases, grading protocols, and subset definitions are in Appendix D.

Table 1: Main results on four benchmarks. Top left: HLE-Verified (Accuracy). Top right: LiveCodeBench (Pass@1, overall and by difficulty). Bottom left: GPQA-Diamond (Accuracy). Bottom right: BabyVision (Avg@3, overall and by task family). Each subtable shows zero-shot model references (upper) and test-time scaling methods (lower). **Bold** marks the best per group; lowest $/q among test-time methods is also bolded.

(a) HLE-Verified

	Acc. (%)	$/q
Zero-Shot Models		
Gemini 3 Pro	48.93	–
GPT-5.2-High	**52.48**	–
Claude Opus 4.5	48.16	–
Grok 4.1 Fast Reasoning	43.07	–
Claude Opus 4.6	50.16	–
DeepSeek-V3.2	47.45	–
Qwen3-Max-Thinking	48.48	–
Test-Time Scaling Methods		
Pass@1	48.00	**0.28**
Self-Refine	53.00	0.41
Budget Forcing	51.00	0.60
Skywork-Reward-V2	52.00	2.22
LLM Selection (N=8)	**58.00**	4.53
ATTS (Ours)	56.00	1.90
ATTS Multi-model Med	57.00	1.63

(b) LiveCodeBench

	Pass@1	Easy	Med.	Hard	$/q
Zero-Shot Models					
O4-Mini (High)	**80.2**	**99.1**	**89.4**	**63.5**	–
O3 (High)	75.8	**99.1**	84.4	57.1	–
O4-Mini (Med.)	74.2	98.2	86.5	52.7	–
Gemini-2.5-Pro	73.6	**99.1**	87.2	50.2	–
DeepSeek-R1	73.1	98.7	85.2	50.7	–
Test-Time Scaling Methods					
Pass@1	77.14	95.35	88.46	60.00	**0.24**
Self-Refine	80.57	**100.00**	90.38	63.75	0.32
Budget Forcing	80.00	**100.00**	88.46	63.75	0.50
Skywork-Reward-V2	78.86	**100.00**	90.38	60.00	1.41
LLM Selection (N=8)	81.14	**100.00**	86.54	67.50	1.51
ATTS (Ours)	82.29	**100.00**	90.38	67.50	0.53
ATTS Multi-model Med	**85.63**	**100.00**	**92.31**	**73.42**	0.69

(c) GPQA-Diamond

	Acc. (%)	$/q
Zero-Shot Models		
Gemini 3.1 Pro Preview	**94.1**	–
GPT-5.4 (xhigh)	92.0	–
GPT-5.3 Codex (xhigh)	91.5	–
Gemini 3 Pro Preview (high)	90.8	–
GPT-5.2 (xhigh)	90.3	–
Test-Time Scaling Methods		
Pass@1	74.24	**0.13**
Majority Voting	74.24	0.90
Self-Refine	72.22	0.23
Budget Forcing	76.77	0.42
Skywork-Reward-V2	75.76	0.90
LLM Selection (N=8)	77.27	0.97
ATTS (Ours)	**82.83**	0.35
ATTS Multi-model Med	81.82	0.39

(d) BabyVision

	Overall	Fine-gr.	Tracking	Spatial	Pattern	$/q
Human	94.1	92.3	94.6	94.7	97.8	–
Zero-Shot Models						
Gemini3-Pro-Preview	**49.7**	**46.2**	**43.4**	**53.7**	53.9	–
GPT-5.2	34.4	27.3	34.9	35.2	**54.9**	–
Doubao-1.8	30.2	39.2	15.7	24.7	37.7	–
Qwen3-VL-Plus	19.2	21.8	11.5	18.1	25.5	–
Grok-4	16.2	11.0	24.1	13.2	24.8	–
Test-Time Scaling Methods						
Pass@1	19.59	20.86	19.28	19.78	15.69	**0.07**
Majority Voting	22.42	**23.93**	21.69	23.08	**23.53**	0.50
Self-Refine	21.39	20.25	22.89	20.88	**23.53**	0.42
Budget Forcing	21.91	20.86	**28.92**	21.98	13.73	0.52
VisualPRM	20.88	20.86	21.69	21.98	17.65	0.50
LLM Selection (N=8)	**23.20**	23.31	25.30	23.08	19.61	0.56
ATTS (Ours)	**23.20**	22.09	24.10	**26.37**	19.61	0.27
ATTS Multi-model Med	22.42	22.09	27.71	21.98	15.69	0.43

HLE-Verified. On HLE-Verified, ATTS achieves 56.00% accuracy at $1.90/q, outperforming Self-Refine (53.00%) and Budget Forcing (51.00%). LLM Selection reaches 58.00% but at $4.53/q—a $2.4\times$ cost increase for a 2-point gain—illustrating a cost-accuracy tradeoff where single-pass aggregation can achieve higher accuracy when cost is unconstrained, but adaptive exploration provides a more efficient operating point. The multi-model variant improves to 57.00% at $1.63/q, narrowing the accuracy gap while maintaining cost efficiency.

LiveCodeBench. For LiveCodeBench, the single-model ATTS configuration reaches 82.29% Pass@1 at $0.53/q, improving over Self-Refine (80.57%) and Budget Forcing (80.00%). The gains concentrate on Hard problems: 67.50% vs. 63.75% for both Self-Refine and Budget Forcing, while Easy (100%) and Medium (90.38%) accuracies are at or near ceiling. The multi-model variant further improves to 85.63% overall and 73.42% on Hard.

GPQA-Diamond. On GPQA-Diamond, ATTS achieves the highest accuracy among all test-time scaling methods (82.83%) at $0.35/q, substantially outperforming Budget Forcing (76.77%) and Skywork-Reward-V2 (75.76%). Self-Refine (72.22%) falls below Pass@1 (74.24%), consistent with the observation that its feedback model frequently accepts the initial draft without revision, and occasionally introduces errors when it does revise.

BabyVision. Among test-time scaling methods, ATTS leads on BabyVision with 23.20% overall accuracy at $0.27/q, ahead of Majority Voting (22.42%) and Budget Forcing (21.91%). The largest per-subset margin appears on Spatial Perception, where ATTS reaches 26.37%.

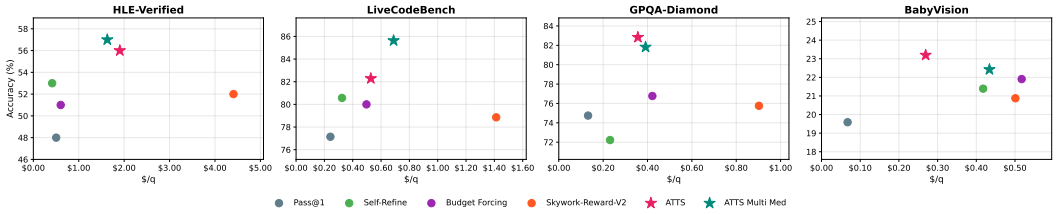

Figure 1: Cost-accuracy tradeoffs across the four main benchmarks (left to right: HLE-Verified, LiveCodeBench, GPQA-Diamond, BabyVision). Star markers denote ATTS variants; circles denote baselines. ATTS achieves competitive or superior accuracy at lower cost across all benchmarks.

Overall trends. Across benchmarks, ATTS is intended to occupy a middle ground between fixed-budget sequential refinement and purely parallel sampling. Direct one-shot generation is often already strong on easier exact-match tasks, but it can leave performance on the table once problems require either broader hypothesis coverage or evidence reconciliation across partially correct candidates. Majority Voting addresses the first issue by increasing candidate diversity, yet it still reduces the final decision to selection. The role of ATTS is to use the same test-time budget more adaptively: explore when the current pool is incomplete or inconsistent, and synthesize the final answer once the candidate set contains enough evidence.

This framing is especially useful on benchmarks where independently generated candidates can expose complementary reasoning fragments rather than merely redundant answers. On mathematical and scientific reasoning tasks, the expected gain comes from avoiding premature commitment to a single trajectory while still permitting a model-based final consolidation step. On code and multimodal reasoning tasks, the same controller provides a unified way to trade off breadth against synthesis without changing the benchmark-specific grading pipeline. Taken together, the results support the main claim of the paper: even a purposefully constrained single-tool interface can be sufficient to turn test-time scaling into a semantic compute-allocation problem rather than a fixed sampling schedule.

4.3 Ablation Studies

Exploration effort. The multi-model variant exposes an *exploration effort* parameter that controls stopping aggressiveness. Figure 2 shows the cost-accuracy tradeoff at three effort levels (Low, Medium, High) across all four benchmarks, holding the model pool and budget fixed; the raw numbers are in Appendix C, Table 4. The Low-to-Medium transition yields the largest accuracy gain on every benchmark (up to +13 points on HLE, +8.6 on LiveCodeBench). Medium-to-High provides diminishing returns: on LiveCodeBench and BabyVision accuracy is flat while cost nearly doubles; only on HLE-Verified does High effort produce a meaningful further improvement (+3 points). Medium effort therefore represents the best cost-efficiency operating point across the benchmark suite.

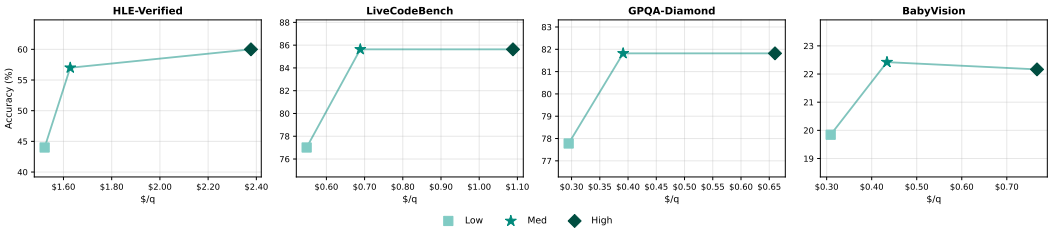

Figure 2: Effect of exploration effort on multi-model ATTS. Each panel shows the cost-accuracy tradeoff for Low, Medium, and High effort on one benchmark. The steep Low-to-Medium gain and flat Medium-to-High plateau are consistent across all benchmarks.

Adding a separate integrator. In the default ATTS architecture, the orchestrator produces the final answer directly from its accumulated evidence after deciding to stop exploring. We also evaluate a variant that adds a dedicated `integrate` step: a separate model call that receives all candidates and synthesizes the final answer independently.

Table 2: Effect of adding a separate integrator. The default ATTS architecture has the orchestrator synthesize directly; the "+ Integrator" variant delegates synthesis to a separate model call.

	Default (no integrator)		+ Integrator	
	Acc. (%)	$/q	Acc. (%)	$/q
HLE-Verified	56.00	1.90	57.00	2.66
LiveCodeBench	82.29	0.53	80.57	0.72
BabyVision	23.20	0.27	23.20	0.34
GPQA-Diamond	82.83	0.35	76.26	0.44

Across all four benchmarks, adding a separate integrator does not consistently improve accuracy and always increases cost (Table 2). On GPQA-Diamond, accuracy drops from 82.83% to 76.26%; on LiveCodeBench, from 82.29% to 80.57%. On HLE-Verified, the integrator variant gains 1 point (57.00% vs. 56.00%) but at substantially higher cost ($2.66/q vs. $1.90/q). The degradation likely stems from the integrator having less context for synthesis: it only sees the final candidate pool in a single-turn call, whereas the orchestrator has observed the full multi-turn conversation including each candidate's arrival and the reasoning behind its own exploration decisions.

Orchestrator model selection. We ablate the orchestrator model while keeping the explorer fixed at Sonnet 4.6. Figure 3 shows accuracy and cost for Haiku 4.5, Sonnet 4.6, and Opus 4.6 as orchestrator on three benchmarks; the raw numbers are in Appendix C, Table 5. Using Haiku 4.5 as orchestrator reduces per-call orchestrator cost but degrades accuracy: 79.80% vs. 82.83% on GPQA, 79.43% vs. 82.29% on LiveCodeBench. The accuracy loss is not solely due to weaker synthesis. Haiku dispatches significantly more explore calls on average (3.14 vs. 1.85 on LiveCodeBench), indicating it is less confident in its stopping decisions and compensates by requesting more evidence. The additional explore calls increase the total explorer cost, which more than offsets the savings from a cheaper orchestrator. This suggests that the orchestrator role demands strong meta-reasoning ability: a cheaper model saves on per-call cost but makes systematically worse allocation decisions. Figure 3 visualizes this tradeoff.

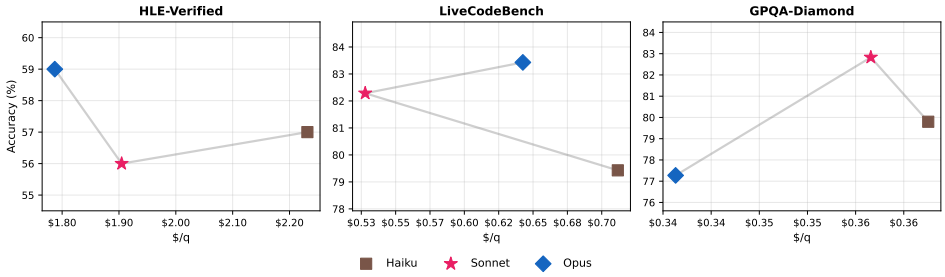

Figure 3: Effect of orchestrator model on ATTS accuracy and cost. Haiku is cheapest per call but over-explores, increasing total cost. Opus improves accuracy on HLE and LiveCodeBench but not on GPQA.

5 Limitations

ATTS inherits several limitations from the underlying language model and from its own orchestrator design. First, the method can only synthesize evidence that is already present in the candidate pool, so if all explored candidates share the same misconception, the orchestrator has little opportunity to recover. Second, the current framework does not provide formal guarantees for stopping optimality, calibration, or budget efficiency; the orchestrator is still a learned heuristic policy, even if its action space is intentionally simple. In practice, we observe that the orchestrator tends to converge early: on GPQA, 88% of questions use exactly two `explore` calls before stopping, even though the oracle best-of-N curve continues to climb through $N{=}8$. This conservative exploration policy leaves headroom on the table, and learning a better stopping criterion remains an open problem. Third, because ATTS requires multiple model calls, it can increase latency and inference cost relative to direct generation on easy problems where extra exploration is unnecessary. Finally, the current evaluation focuses on benchmarked reasoning tasks with fixed answer interfaces rather than on open-ended tool-use agents, so the present results should be interpreted as evidence about reasoning-oriented test-time scaling rather than as a complete account of agentic decision making.

6 Conclusion

The central empirical finding of this work is that a minimal action space—a single `explore` tool with adaptive stopping—is sufficient to match or outperform more complex test-time scaling strategies across diverse reasoning domains while remaining cost-competitive. A multi-model variant further improves to 57.00% and 85.63% on HLE-Verified and LiveCodeBench respectively. The ablation results sharpen this finding: removing the orchestrator's multi-turn context by delegating synthesis to a separate call consistently degrades accuracy, demonstrating that stateful evidence management—not just candidate diversity—drives the gains. For the broader field, these results suggest that adaptive compute allocation may be more productively framed as a sequential decision problem over candidate hypotheses than as explicit tree search or fixed sampling. A key limitation is the orchestrator's conservative stopping behavior: on GPQA-Diamond, 88% of questions use exactly two `explore` calls, leaving substantial headroom relative to oracle best-of-N. Learning stopping policies that better exploit the full exploration budget remains an open problem.

References

[1] Anthropic. Claude code SDK documentation. `https://docs.anthropic.com/en/docs/claude-code/sdk`, 2025.

[2] Center for AI Safety, Scale AI, and HLE Contributors Consortium. A benchmark of expert-level academic questions to assess AI capabilities. *Nature*, 649:1139–1146, 2026. doi: 10.1038/s41586-025-09962-4.

[3] Feng Chen, Allan Raventos, Nan Cheng, Surya Ganguli, and Shaul Druckmann. Rethinking fine-tuning when scaling test-time compute: Limiting confidence improves mathematical

reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.07154.

[4] Liang Chen, Weichu Xie, Yiyan Liang, et al. Babyvision: Visual reasoning beyond language, 2026. URL https://arxiv.org/abs/2601.06521.

[5] Yongchao Chen, Jiefeng Chen, Rui Meng, Ji Yin, Na Li, Chuchu Fan, Chi Wang, Tomas Pfister, and Jinsung Yoon. Tumix: Multi-agent test-time scaling with tool-use mixture, 2025. URL https://arxiv.org/abs/2510.01279.

[6] Audrey Huang, Adam Block, Qinghua Liu, Nan Jiang, Akshay Krishnamurthy, and Dylan J. Foster. Is best-of-n the best of them? coverage, scaling, and optimality in inference-time alignment. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2503.21878.

[7] Idavidrein. Idavidrein/gpqa (hugging face dataset card). https://huggingface.co/datasets/Idavidrein/gpqa, 2026.

[8] Naman Jain, Kevin Han, Alex Gu, et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *ICLR*, 2025. URL https://arxiv.org/abs/2403.07974.

[9] Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Agentic test-time scaling for webagents, 2026. URL https://arxiv.org/abs/2602.12276.

[10] Xiaochuan Li, Ryan Ming, Pranav Setlur, Abhijay Paladugu, Andy Tang, Hao Kang, Shuai Shao, Rong Jin, and Chenyan Xiong. Benchmark test-time scaling of general LLM agents, 2026. URL https://arxiv.org/abs/2602.18998.

[11] Chris Yuhao Liu, Liang Zeng, Yuzhen Xiao, Jujie He, Jiacai Liu, Chaojie Wang, Rui Yan, Wei Shen, Fuxiang Zhang, Jiacheng Xu, Yang Liu, and Yahui Zhou. Skywork-reward-v2: Scaling preference data curation via human-ai synergy, 2025. URL https://arxiv.org/abs/2507.01352.

[12] Xiang Liu, Xuming Hu, Xiaowen Chu, and Eunsol Choi. Diffadapt: Difficulty-adaptive reasoning for token-efficient LLM inference, 2025. URL https://arxiv.org/abs/2510.19669.

[13] livecodebench. livecodebench/code_generation_lite (hugging face dataset card). https://huggingface.co/datasets/livecodebench/code_generation_lite, 2026.

[14] LiveCodeBench Team. Livecodebench leaderboard. https://livecodebench.github.io/leaderboard.html, 2026.

[15] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf.

[16] MathArena. Matharena competition table: AIME 2025. https://matharena.ai/competition_tables/aime--aime_2025, 2026.

[17] MathArena. Matharena competition table: AIME 2026. https://matharena.ai/competition_tables/aime--aime_2026, 2026.

[18] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20275–20321. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1025. URL https://aclanthology.org/2025.emnlp-main.1025/.

[19] OpenGVLab. Visualprm-8b (hugging face model card). https://huggingface.co/OpenGVLab/VisualPRM-8B, 2025.

[20] David Rein, Betty Li Hou, Asa Cooper Stickland, et al. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2311.12022.

[21] Stuart J. Russell and Eric Wefald. *Do the Right Thing: Studies in Limited Rationality*. MIT Press, Cambridge, MA, 1991.

[22] skylenage. skylenage/hle-verified (hugging face dataset card). https://huggingface.co/datasets/skylenage/HLE-Verified, 2026.

[23] Skywork. Skywork-reward-v2-qwen3-8b (hugging face model card). https://huggingface.co/Skywork/Skywork-Reward-V2-Qwen3-8B, 2025.

[24] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2408.03314.

[25] Richard Sutton. The bitter lesson. http://www.incompleteideas.net/IncIdeas/BitterLesson.html, 2019.

[26] Guiyao Tie, Zenghui Yuan, Zeli Zhao, Chaoran Hu, Tianhe Gu, Ruihang Zhang, Sizhe Zhang, Junran Wu, Xiaoyue Tu, Ming Jin, Qingsong Wen, Lixing Chen, Pan Zhou, and Lichao Sun. Can LLMs correct themselves? a benchmark of self-correction in LLMs, 2025. URL https://arxiv.org/abs/2510.16062.

[27] UniPat-AI. Unipatai/babyvision (hugging face dataset card). https://huggingface.co/datasets/UnipatAI/BabyVision, 2026.

[28] Siddarth Venkatraman, Vineet Jain, Sarthak Mittal, Vedant Shah, Johan Obando-Ceron, Yoshua Bengio, Brian R. Bartoldson, Bhavya Kailkhura, Guillaume Lajoie, Glen Berseth, Nikolay Malkin, and Moksh Jain. Recursive self-aggregation unlocks deep thinking in large language models, 2025. URL https://arxiv.org/abs/2509.26626.

[29] Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Yiran Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2312.08935.

[30] Weiyun Wang, Zhangwei Gao, Lianjie Chen, Zhe Chen, Jinguo Zhu, Xiangyu Zhao, Yangzhou Liu, Yue Cao, Shenglong Ye, Xizhou Zhu, Lewei Lu, Haodong Duan, Yu Qiao, Jifeng Dai, and Wenhai Wang. Visualprm: An effective process reward model for multimodal reasoning, 2025. URL https://arxiv.org/abs/2503.10291.

[31] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2203.11171.

[32] Yuxi Xie, Anirudh Goyal, Wenyue Zheng, Min-Yen Kan, Timothy P. Lillicrap, Kenji Kawaguchi, and Michael Shieh. Monte carlo tree search boosts reasoning via iterative preference learning, 2024. URL https://arxiv.org/abs/2405.00451.

[33] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2305.10601.

[34] Weiqi Zhai, Zhihai Wang, Jinghang Wang, et al. Hle-verified: A systematic verification and structured revision of humanity's last exam, 2026. URL https://arxiv.org/abs/2602.13964.

A Implementation Details

Controller and backend setup. ATTS is implemented as a delegated controller with two roles: an orchestrator and an explorer. The orchestrator never solves the task directly; it only decides whether to spend the remaining budget on another `explore` call or to stop and synthesize the final answer from the accumulated evidence. Each `explore` call returns a structured candidate containing an answer, supporting reasoning, an approach summary, and an optional confidence score. When the orchestrator decides to stop, it produces the final answer directly via the SDK's structured output mechanism. For the Claude implementation, we use the Claude Agent SDK [1] and expose `explore` as the only available delegated tool. We also evaluate a variant that adds a separate `integrate` tool that delegates final synthesis to a dedicated model call; see the ablation in Section 4.4. The same high-level controller is reused across benchmarks, while the underlying sub-model backend can be swapped without changing the controller logic.

Prompt families. ATTS uses two prompt families: a benchmark-agnostic orchestrator prompt and a benchmark-specific explorer prompt. The orchestrator prompt defines the controller role and explicitly instructs it not to solve the task itself. The explorer prompt asks for an independent solution from scratch under the target answer format. Representative prompt excerpts are shown in Listings 1 and 3. For Claude runs, these prompts are paired with structured-output schemas so that each sub-call returns machine-readable fields rather than free-form text only.

```
You are a meta-reasoning orchestrator. You manage a pool of candidate solutions for a
     problem.

After each explore, you will see the candidate result (answer, confidence, approach,
     reasoning).

Your final answer must be derived from candidate outputs. You may combine insights from
     multiple candidates, but you cannot introduce information that no candidate provided
     .

## Principles (HIGHEST PRIORITY)

1. You cannot solve problems yourself. Your only window into the problem is what solvers
     return. Reasoning about the problem content constitutes solving.
2. A single candidate, regardless of confidence, does not constitute sufficient evidence.
      Self-reported confidence is poorly calibrated.
3. Genuine convergence means independent solvers arriving at the same answer through
     different methods. Shared reasoning paths may reflect shared misconceptions.
4. Solver failure (timeout, empty answer) reflects problem difficulty. Repeated failures
     mean the problem is practically unsolvable.
5. CRITICAL: Each explore costs budget. Giving up is valid and preferred -- submit empty
     answer rather than wasting budget.

[6 worked examples follow, showing: starting exploration, disagreement resolution,
     timeout handling, escalation, and giving up.]
```

Listing 1: Orchestrator system prompt (single-model, explore-only variant). The same principles and examples are shared by the integrate and multi-model variants; only the finalize instruction differs.

```
You have access to multiple solver models of different capability and cost. Each explore
     call lets you choose which model to use.

## Model Profiles
Pricing (per million tokens): Haiku $1/$5, Sonnet $3/$15, Opus $5/$25.
[Public benchmark scores table: MATH-500, AIME, GPQA, HLE, ARC-AGI-2, SWE-bench, etc.]

## Adapted Principles
3. Candidates from different models that agree provide stronger evidence than same-model
     candidates.
4. A weaker model failing does not mean a stronger model will also fail. Start cheap,
     escalate on failure.

## Exploration Effort Constraints (appended to user message)
LOW:    Stop as soon as 2 candidates from any model converge.
MEDIUM: MUST call explore >= 3 times, MUST use >= 2 models.
HIGH:   MUST call explore >= 5 times, MUST use all 3 models.
```

Listing 2: Multi-model orchestrator prompt additions. The orchestrator receives model profiles with pricing and benchmark scores, and adapted principles for heterogeneous model selection.

```
Explorer:
You are an expert problem solver. Solve the given problem step by step.
If you cannot solve it exactly, give your best estimate and set confidence accordingly.
Schema: {approach, reasoning, answer, confidence}

Integrator:
You are an expert answer synthesizer. You will receive a problem and multiple candidate
     solutions.
Your job: (1) Analyze each candidate's reasoning individually, (2) Identify agreements
     and disagreements, (3) Judge which side has more rigorous reasoning, (4) Point out
     logical errors, (5) Produce a verified final answer.
Schema: {analysis, final_answer, reasoning}
```

Listing 3: Explorer prompt (left) and integrator prompt (right). Both share the same answer format rules enforced via structured output schema field descriptions.

Schemas and benchmark-specific prompt instantiation. Benchmark-specific adaptation is intentionally lightweight. Most answer-based tasks share the explorer schema `{approach, reasoning, answer, confidence}` and the integrator schema `{analysis, final_answer, reasoning}`. Only the answer interface changes across tasks. LiveCodeBench uses code-generation fields, AIME requires an integer final answer, GPQA expects one option letter, and multimodal benchmarks pass the associated image with the text question. This design keeps the controller fixed while preserving each benchmark's native answer format and grading protocol.

Runtime behavior. At runtime, the controller maintains the candidate pool and repeatedly calls `explore` until the exploration budget is exhausted or the controller decides to stop and synthesize. The same implementation supports both text-only and image-conditioned tasks, and all runs can optionally record the controller trace and sub-model outputs for later inspection. These implementation choices are meant to make the evaluation pipeline inspectable without changing the central adaptive exploration decision process.

Grading. LiveCodeBench and AIME use deterministic grading (code execution and exact integer matching, respectively). GPQA-Diamond uses exact option-letter matching. HLE-Verified and BabyVision use an LLM judge (Claude Haiku 4.5) that compares the predicted answer against the reference answer.

Baseline implementations. All baselines use the same backbone model and the same benchmark-specific answer format as ATTS, so the comparison isolates the inference-time strategy.

Pass@1 generates a single independent response per question and returns its answer directly.

Majority Voting draws $N = 8$ independent responses and selects the most frequent answer.

Self-Refine [15] generates an initial draft, then enters a feedback-revision loop: a separate feedback call critiques the current draft and returns a correctness judgment; if the draft is judged incorrect, a revision call produces an updated draft conditioned on all prior drafts and feedbacks. The loop terminates when feedback returns a positive judgment or the iteration budget is exhausted. In practice, the feedback model frequently accepts the initial draft, so Self-Refine terminates after a single feedback call on the majority of questions, which explains its relatively low cost on most benchmarks.

Budget Forcing [18] runs N sequential rounds on the same question. Round 1 generates an initial solution. Each subsequent round feeds the previous round's full reasoning trace back to the model with "Wait" appended, suppressing premature termination and forcing the model to continue deliberating before producing a new answer. All N rounds run unconditionally, and the final round's answer is returned. Because continuation rounds extend the existing reasoning rather than solving from scratch, they tend to produce shorter completions and lower per-round cost.

Reward-model reranking (Skywork-Reward-V2 for text benchmarks, VisualPRM for multimodal benchmarks) draws the same $N = 8$ independent candidates, then scores each with a separately trained reward model and returns the highest-scoring answer. Skywork-Reward-V2 is an outcome reward model that assigns a single scalar score per response; VisualPRM is a process reward model that aggregates per-step scores. The reward models run locally and incur no API cost, but the candidate generation cost is identical to that of Majority Voting.

B AIME Results

AIME 2025 and AIME 2026 are 30-problem final-answer mathematics benchmarks with exact integer-answer grading. Because the backbone model already achieves near-ceiling accuracy on these benchmarks (93.33% on AIME 2025 with a single attempt), the statistical resolution for method comparison is limited. We include them here for completeness.

Table 3 presents the AIME 2025 and AIME 2026 results side by side. Full benchmark details including the official answer-space constraint, source repositories, and leaderboard alignment are in Appendix, AIME 2025 and Appendix, AIME 2026.

Table 3: AIME results. Left: AIME 2025. Right: AIME 2026. Both are 30-problem final-answer benchmarks with exact integer-answer grading. Cited from official online records.

(a) AIME 2025

	Acc. (%)	$/q
Zero-Shot Models		
GPT-5.2 (high)	**100.00**	–
Step 3.5 Flash	98.33	–
Gemini 3 Flash	97.50	–
GLM 5	96.67	–
DeepSeek-v3.2-Speciale	95.83	–
Test-Time Scaling Methods		
Pass@1	**93.33**	**0.21**
Majority Voting	**93.33**	1.49
Self-Refine	**93.33**	3.13
Budget Forcing	**93.33**	0.52
Skywork-Reward-V2	90.00	1.49
ATTS (Ours)	**93.33**	0.43

(b) AIME 2026

	Acc. (%)	$/q
Zero-Shot Models		
GPT-5.4 (xhigh)	**99.17**	–
GPT-5.2 (high)	98.33	–
Gemini 3.1 Pro Preview	98.33	–
Step 3.5 Flash	96.67	–
Claude-Opus-4.6 (High)	96.67	–
Test-Time Scaling Methods		
Pass@1	93.33	**0.22**
Majority Voting	93.33	1.37
Self-Refine	90.00	0.23
Budget Forcing	93.33	0.60
Skywork-Reward-V2	**96.67**	1.37
ATTS (Ours)	**96.67**	0.48

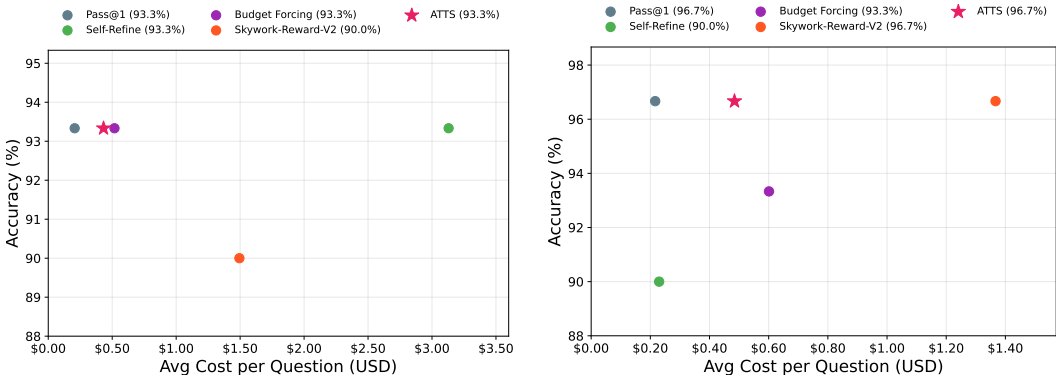

Figure 4: Cost-accuracy tradeoffs on AIME 2025 (left) and AIME 2026 (right).

AIME 2025. On AIME 2025, most test-time scaling methods converge to the same accuracy (93.33%), with only Skywork-Reward-V2 falling behind at 90.00%. The 30-problem dataset leaves little statistical room for differentiation, and the backbone model already solves 28 of 30 problems with a single attempt.

AIME 2026. On AIME 2026, ATTS and Skywork-Reward-V2 both reach 96.67%, improving over Pass@1 (93.33%). Self-Refine drops to 90.00%, likely because its feedback model misjudges some correct solutions and introduces errors during revision. As with AIME 2025, the small dataset size (30 problems) limits the granularity of method comparisons.

C Ablation Data

Table 4: Effect of exploration effort on multi-model ATTS. Accuracy (%) and average cost per question ($/q) across benchmarks.

	Low		Medium		High	
	Acc.	$/q	Acc.	$/q	Acc.	$/q
HLE-Verified	44.00	1.52	57.00	1.63	60.00	2.38
LiveCodeBench	77.01	0.55	85.63	0.69	85.63	1.09
BabyVision	19.85	0.31	22.42	0.43	22.16	0.77
GPQA-Diamond	77.78	0.29	81.82	0.39	82.32	0.40

Table 5: Effect of orchestrator model on ATTS accuracy (%) and cost ($/q). Explorer is Sonnet 4.6 throughout.

	Haiku orch		Sonnet orch		Opus orch	
	Acc.	$/q	Acc.	$/q	Acc.	$/q
HLE-Verified	57.00	2.23	56.00	1.90	59.00	1.79
LiveCodeBench	79.43	0.71	82.29	0.53	83.43	0.64
GPQA-Diamond	79.80	0.36	82.83	0.35	77.27	0.34

D Benchmark Details

HLE-Verified. We use the official HLE-Verified dataset release `skylenage/HLE-Verified` [22], which partitions the 2,500 audited HLE items into Gold (668), Revision (1,143), and Uncertain (689) subsets [34]. HLE-Verified preserves the original HLE answer-based evaluation semantics: each item contains a problem statement, a final answer, and sometimes a rationale, but official scoring is driven by the problem and final answer, with rationale used only as auxiliary evidence during verification and revision [2, 34]. In the HLE-Verified evaluation paper, the reported baselines are run on the text-only portion of HLE using the official HLE system prompt, each model's default recommended decoding configuration, and five independent rollouts per item; the paper reports both *avg5 Accuracy* and *Calibration Error*, with calibration computed by the official HLE evaluation code [34]. In this paper, HLE reporting is aligned to the Gold-subset setting, while Table 1(a) reproduces the official revised-subset accuracy references from Zhai et al. [34] for context. Our grading uses an LLM-as-judge pipeline with Claude Haiku 4.5 as the judge model, which determines semantic equivalence between the predicted and gold answers.

LiveCodeBench. We use the LiveCodeBench code generation release `livecodebench/code_generation_lite` [13], configuration `v6` [8]. LiveCodeBench is a continuously updated programming benchmark sourced from LeetCode, AtCoder, and CodeForces. Each problem is tagged by release date, so official evaluations can be restricted to post-cutoff time windows for contamination-aware comparison [8]. In the official code generation setting, the model receives the natural-language problem statement together with example input-output pairs and must return a complete executable program. Correctness is determined purely by functional correctness on unseen tests, using the benchmark harness rather than an LLM judge [8]. The benchmark paper reports *Pass@1* as the primary metric and also reports difficulty-stratified *Pass@1* using the platform-provided Easy/Medium/Hard labels. The public generation leaderboard [14] follows this same framing, and Table 1(b) therefore reports overall and difficulty-wise Pass@1 from the same leaderboard view. Our grading uses the official `lcb_runner` execution harness with a 10-second per-test-case timeout.

BabyVision. We use the BabyVision dataset release `UnipatAI/BabyVision` [27], which contains 388 image-question pairs spanning four task families: Fine-grained Discrimination (163), Visual Tracking (83), Spatial Perception (91), and Visual Pattern Recognition (51) [4]. The benchmark contains 135 multiple-choice items and 253 fill-in-the-blank items; in the official inference setup,

each model is queried with a unified prompt template of the form "`{question} Think about the question and give your final answer in {format}.`", which standardizes answer formatting for downstream extraction and judging [4]. Official scoring uses an LLM-as-judge pipeline: given a model answer and the gold answer, Qwen3-Max determines semantic equivalence under the benchmark's fixed judge prompt, which the authors report as having perfect agreement with their human evaluators on the benchmark responses they audited [4]. The primary metric is *Avg@3*, defined in the paper as the mean *Pass@1* accuracy across three random runs, and the official tables additionally report the corresponding standard deviation; Table 1(c) follows the official overall-plus-family breakdown but keeps only the mean scores in the main text table. Our grading uses an LLM-as-judge pipeline with Claude Haiku 4.5 as the judge model, following the same semantic equivalence protocol as the official evaluation but substituting the judge model.

AIME 2025. AIME 2025 is treated as a final-answer mathematics benchmark on the MathArena competition table, which lists 30 problems for the 2025 competition and notes that each answer is an integer between 0 and 999 inclusive [16]. The task is constrained final-answer prediction: the model receives the official problem statement and is expected to return one final integer answer in the AIME answer space. Official grading is exact-answer grading against the released answer key, with no LLM judge and no partial credit, so each problem contributes a binary correct/incorrect label. The public leaderboard metric is *Accuracy*, namely the percentage of the 30 official problems answered exactly correctly; Table 3(a) reports that same public leaderboard metric. Our grading uses exact integer matching with no LLM judge, consistent with the official protocol.

AIME 2026. AIME 2026 appears on MathArena as another 30-problem final-answer competition, again with the official note that each answer is an integer between 0 and 999 inclusive [17]. The official task semantics match AIME 2025: the model is given the problem statement and must return one final integer answer. The benchmark performs exact-key grading with no LLM judge and no partial credit, and the leaderboard metric is therefore plain *Accuracy* over the 30 released competition problems. Keeping the same final-answer protocol across 2025 and 2026 allows direct year-to-year comparison under the benchmark's official grading rule; the corresponding public references are shown in Table 3(b). Our grading uses exact integer matching, consistent with the official protocol.

GPQA-Diamond. We use the GPQA release `Idavidrein/gpqa` [7], configuration `gpqa_diamond`, which corresponds to the 198-question Diamond subset introduced in the GPQA paper [20]. GPQA is a graduate-level four-option multiple-choice science benchmark spanning biology, physics, and chemistry, and GPQA-Diamond is the highest-stringency subset retained after expert-accuracy and non-expert-difficulty filtering [20]. The official task is closed-book answer selection unless otherwise stated: the model is shown the question and four candidate options and must output exactly one final option. Scoring is exact option-key accuracy with no semantic judge and a random-guess baseline of 25% [20]. In addition to Diamond-set accuracy, the original GPQA paper also reports accuracies on the Extended and Main sets and distinguishes closed-book few-shot/CoT baselines from an open-book GPT-4-with-search baseline; Table 1(d) keeps the main-text comparison focused on the Diamond-set accuracy column used by current public references. Our grading extracts the option letter (A–D) from the model response via regex and performs exact matching against the gold option key, with no LLM judge, consistent with the official protocol.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [NA]

   Justification: To be completed in the final submission.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [NA]

   Justification: To be completed in the final submission.

3. **Theory Assumptions and Proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [NA]

   Justification: To be completed in the final submission.

4. **Experimental Result Reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [NA]

   Justification: To be completed in the final submission.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [NA]

   Justification: To be completed in the final submission.

6. **Experimental Setting/Details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [NA]

   Justification: To be completed in the final submission.

7. **Experiment Statistical Significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [NA]

   Justification: To be completed in the final submission.

8. **Experiments Compute Resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [NA]

   Justification: To be completed in the final submission.

9. **Code Of Ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [NA]

Justification: To be completed in the final submission.

10. **Broader Impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [NA]

    Justification: To be completed in the final submission.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: To be completed in the final submission.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [NA]

    Justification: To be completed in the final submission.

13. **New Assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [NA]

    Justification: To be completed in the final submission.

14. **Crowdsourcing and Research with Human Subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: To be completed in the final submission.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: To be completed in the final submission.